TASK21: Mini Cicd App

AIM: To develop a CI/CD pipeline simulation application that automates build, test, and deployment steps through a web interface.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for pipeline trigger
4. Capture user action (Run Pipeline)
5. Send request to backend
6. Simulate code checkout step
7. Simulate build process
8. Simulate test execution
9. Simulate deployment step
10. Store pipeline logs
11. Display pipeline status in UI
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
   <title>Mini CI/CD Pipeline</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
```

```
    <h2>CI/CD Pipeline Simulator</h2>
    <button onclick="runPipeline()">Run Pipeline</button>
    <h3>Pipeline Logs</h3>
    <ul id="logs"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
    font-family: Arial;
    background: #1e1e2f;
    color: white;
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}
.container {
    text-align:center;
    width: 400px;
}
button {
    padding: 10px;
    margin: 10px;
    background: #00c6ff;
```

```
    border: none;
    color: white;
    cursor: pointer;
}
li {
    background: #333;
    margin: 5px;
    padding: 5px;
}
```

SCRIPT.JS:

```
function runPipeline() {
    fetch("/run", { method: "POST" })
    .then(() => loadLogs());
}
function loadLogs() {
    fetch("/logs")
    .then(res => res.json())
    .then(data => {
        let list = document.getElementById("logs");
        list.innerHTML = "";
        data.forEach(log => {
            let li = document.createElement("li");
            li.innerText = log;
            list.appendChild(li);
        });
    });
```

```
}
loadLogs();
```

SERVER.JS:

```
const express = require("express");
const db = require("./db");
const app = express();
app.use(express.json());
app.use(express.static(__dirname));
app.post("/run", async (req, res) => {
  await db.runPipeline();
  res.json({ msg: "Pipeline executed" });
});
app.get("/logs", (req, res) => {
  res.json(db.getLogs());
});
app.listen(3000, () => {
  console.log("http://localhost:3000");
});
```

DB.JS:

```
let logs = [];
async function runPipeline() {
  logs = [];
  logs.push("🔄 Code Checkout...");
  await delay();
  logs.push("⚙️ Build Started...");
  await delay();
```

```
  logs.push("🧪 Running Tests...");
  await delay();
  logs.push("✅ Tests Passed");
  await delay();
  logs.push("🚀 Deploying Application...");
  await delay();
  logs.push("🎉 Deployment Successful!");
}
function getLogs() {
  return logs;
}
function delay() {
  return new Promise(resolve => setTimeout(resolve, 1000));
}
module.exports = { runPipeline, getLogs };
```

OUTPUT:

RESULT: A CI/CD pipeline simulation application was successfully developed to automate and visualize build, test, and deployment processes.